

Cloud Networking – Assignment 2

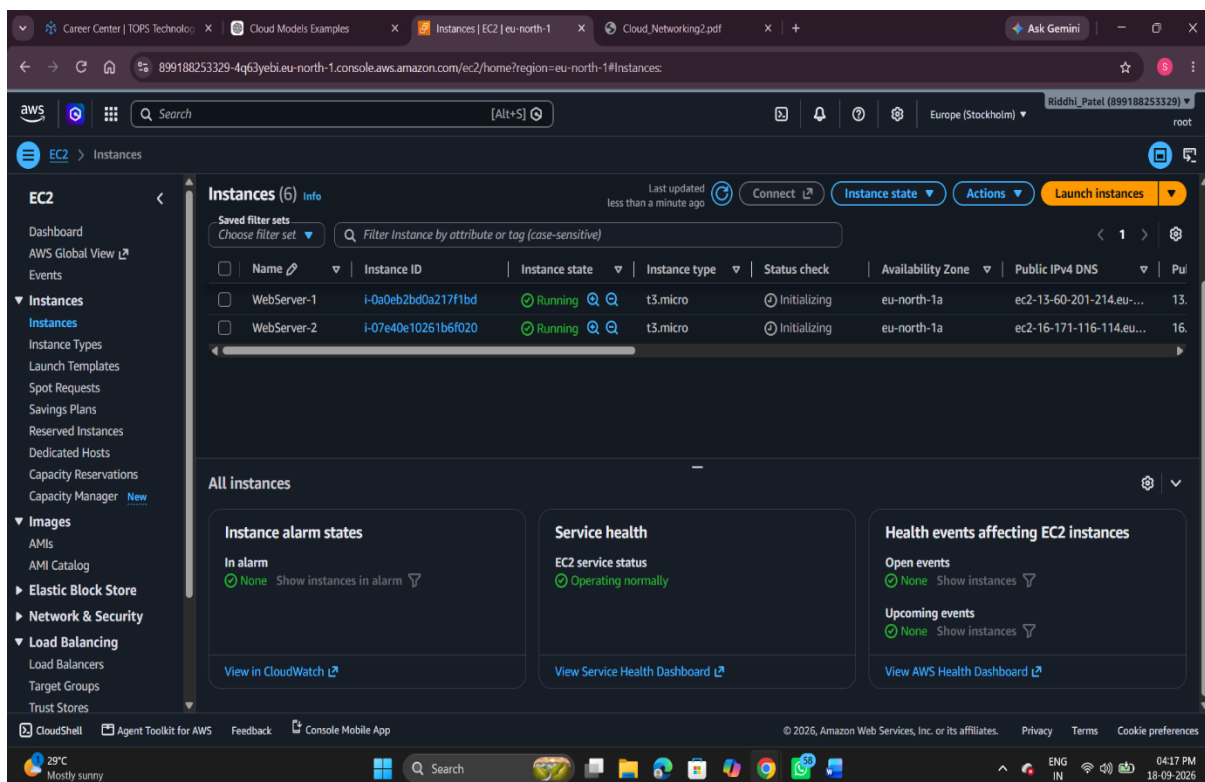
1. In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.

Ans: Steps :

1. Launch two EC2 instances

I launched two EC2 instances named WebServer-1 and WebServer-2.

Screenshot 1: WebServer-1 and WebServer-2 in Running state



2. Connect to WebServer-1 using SSH

I opened the EC2 **Connect** → **SSH client** option and used the provided SSH command in Command Prompt to connect to WebServer-1.

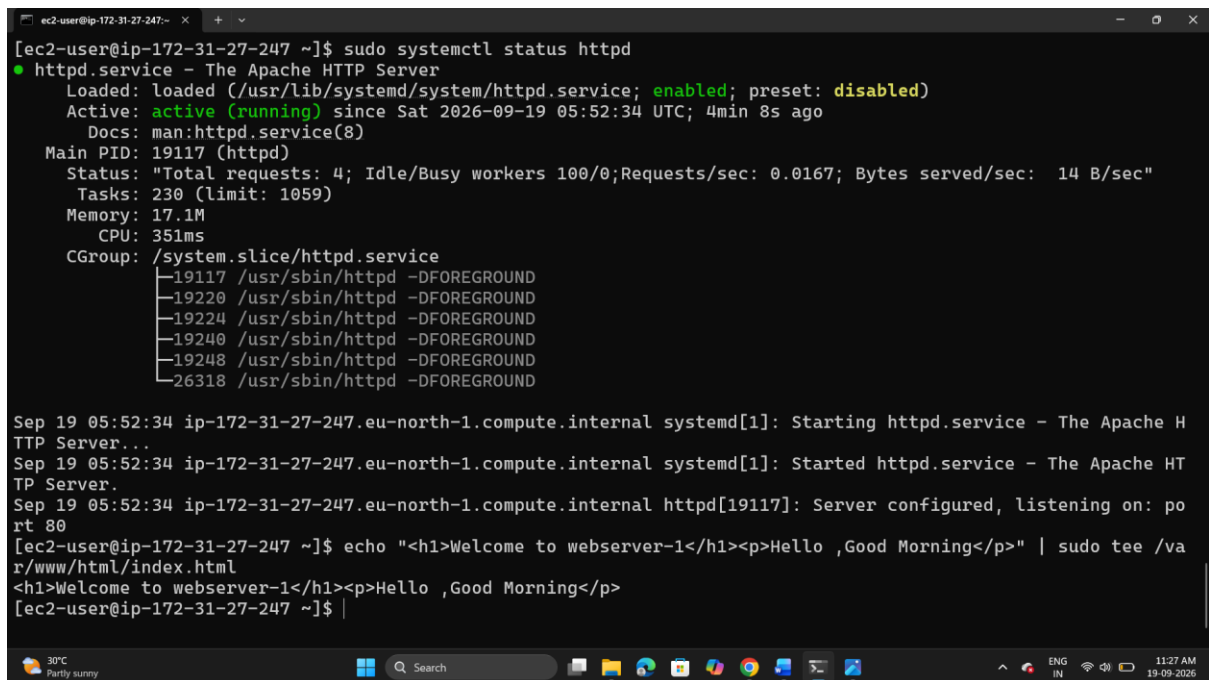
After connecting, I installed and started the Apache web server using:

```
sudo yum install -y httpd
```

```
sudo systemctl start httpd
```

sudo systemctl enable httpd

Screenshot 2: Command Prompt showing successful SSH connection and Apache server running on WebServer-1



```
[ec2-user@ip-172-31-27-247 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-19 05:52:34 UTC; 4min 8s ago
     Docs: man:httpd.service(8)
   Main PID: 19117 (httpd)
   Status: "Total requests: 4; Idle/Busy workers 100/0; Requests/sec: 0.0167; Bytes served/sec: 14 B/sec"
     Tasks: 230 (limit: 1059)
    Memory: 17.1M
       CPU: 351ms
    CGroup: /system.slice/httpd.service
            └─19117 /usr/sbin/httpd -DFOREGROUND
              └─19220 /usr/sbin/httpd -DFOREGROUND
                └─19224 /usr/sbin/httpd -DFOREGROUND
                  └─19240 /usr/sbin/httpd -DFOREGROUND
                    └─19248 /usr/sbin/httpd -DFOREGROUND
                      └─26318 /usr/sbin/httpd -DFOREGROUND

Sep 19 05:52:34 ip-172-31-27-247.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache H
TTP Server...
Sep 19 05:52:34 ip-172-31-27-247.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HT
TP Server.
Sep 19 05:52:34 ip-172-31-27-247.eu-north-1.compute.internal httpd[19117]: Server configured, listening on: po
rt 80
[ec2-user@ip-172-31-27-247 ~]$ echo "<h1>Welcome to webserver-1</h1><p>Hello ,Good Morning</p>" | sudo tee /va
r/www/html/index.html
<h1>Welcome to webserver-1</h1><p>Hello ,Good Morning</p>
[ec2-user@ip-172-31-27-247 ~]$
```

3. Verify WebServer-1

I opened the **public IP address of WebServer-1** in a browser and verified that the webpage was accessible.

Screenshot 3: Browser showing “Welcome from WebServer-1”



4. Connect to WebServer-2 using SSH

I connected to WebServer-2 using its SSH command and installed and started Apache in the same way.

Screenshot 4: Command Prompt showing successful SSH connection and Apache server running on WebServer-2

```
ec2-user@ip-172-31-20-113: ~  
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  
httpd-core-2.4.68-1.amzn2023.0.1.x86_64  
httpd-tools-2.4.68-1.amzn2023.0.1.x86_64  
mailcap-2.1.49-3.amzn2023.0.3.noarch  
mod_lua-2.4.68-1.amzn2023.0.1.x86_64  
httpd-2.4.68-1.amzn2023.0.1.x86_64  
httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch  
libbrotli-1.0.9-4.amzn2023.0.2.x86_64  
mod_http2-2.0.42-1.amzn2023.0.1.x86_64  
  
Complete!  
[ec2-user@ip-172-31-20-113 ~]$ sudo systemctl start httpd  
[ec2-user@ip-172-31-20-113 ~]$ sudo systemctl enable httpd  
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.  
[ec2-user@ip-172-31-20-113 ~]$ echo "<h1>welcome to webserver-2</h1><p>Hello, Good Morning</p>" | sudo tee /var/www/html/index.htmk  
<h1>welcome to webserver-2</h1><p>Hello, Good Morning</p>  
[ec2-user@ip-172-31-20-113 ~]$ echo "<h1>welcome to webserver-2</h1><p>Hello, Good Morning</p>" | sudo tee /var/www/html/index.html  
<h1>welcome to webserver-2</h1><p>Hello, Good Morning</p>  
[ec2-user@ip-172-31-20-113 ~]$ sudo systemctl status httpd  
● httpd.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)  
   Active: active (running) since Sat 2026-09-19 05:59:11 UTC; 1min 58s ago  
     Docs: man:httpd.service(8)  
  Main PID: 20433 (httpd)  
    Status: "Total requests: 3; Idle/Busy workers 100/0;Requests/sec: 0.0275; Bytes served/sec: 23 B/sec"  
    Tasks: 177 (limit: 1059)  
   Memory: 14.0M  
      CPU: 180ms  
   CGroup: /system.slice/httpd.service  
           └─20433 /usr/sbin/httpd -DFOREGROUND  
             └─20534 /usr/sbin/httpd -DFOREGROUND  
               └─20537 /usr/sbin/httpd -DFOREGROUND  
                 └─20538 /usr/sbin/httpd -DFOREGROUND  
                   └─20606 /usr/sbin/httpd -DFOREGROUND  
  
Sep 19 05:59:11 ip-172-31-20-113.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...  
Sep 19 05:59:11 ip-172-31-20-113.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.  
Sep 19 05:59:11 ip-172-31-20-113.eu-north-1.compute.internal httpd[20433]: Server configured, listening on: port 80  
[ec2-user@ip-172-31-20-113 ~]$
```

5. Verify WebServer-2

I opened the **public IP address of WebServer-2** in a browser and verified that the webpage was accessible.

Screenshot 5: Browser showing “Welcome from WebServer-2”



Result

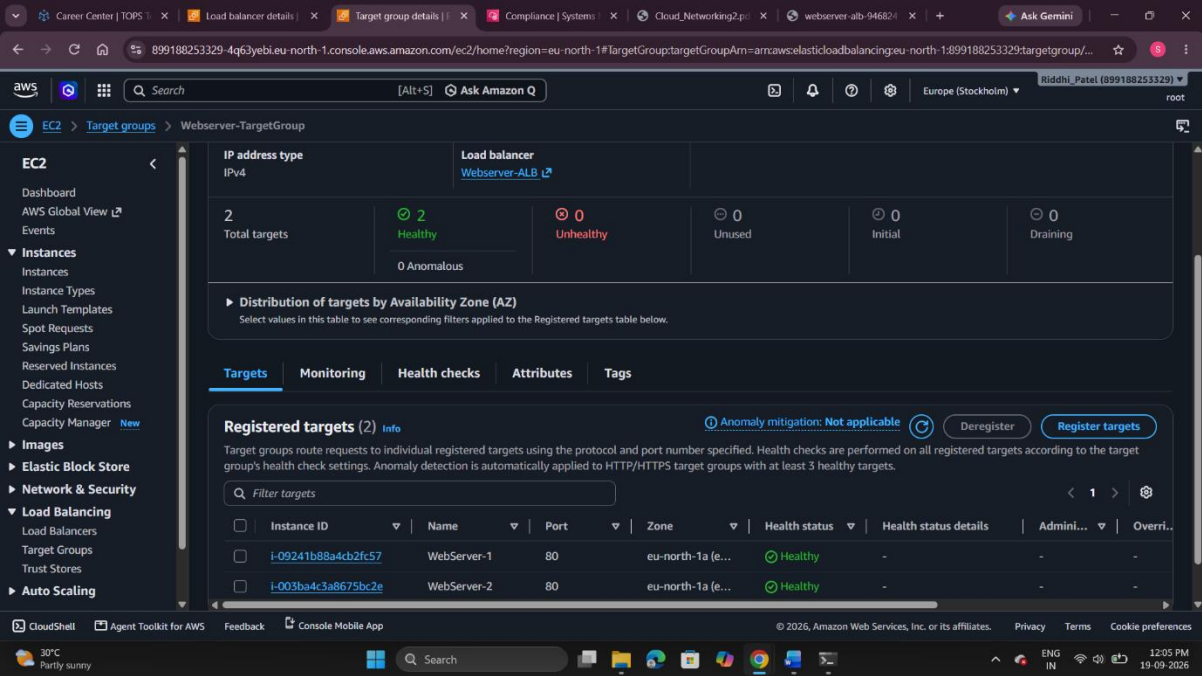
Both EC2 instances are running Apache web servers and both are successfully accessible through their respective public IP addresses.

2. Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

Ans: Step 1: Create Target Group

I created a target group named Webserver-TargetGroup using HTTP on port 80 and registered both EC2 instances, WebServer-1 and WebServer-2.

Screenshot 1 — Target Group with Healthy Targets

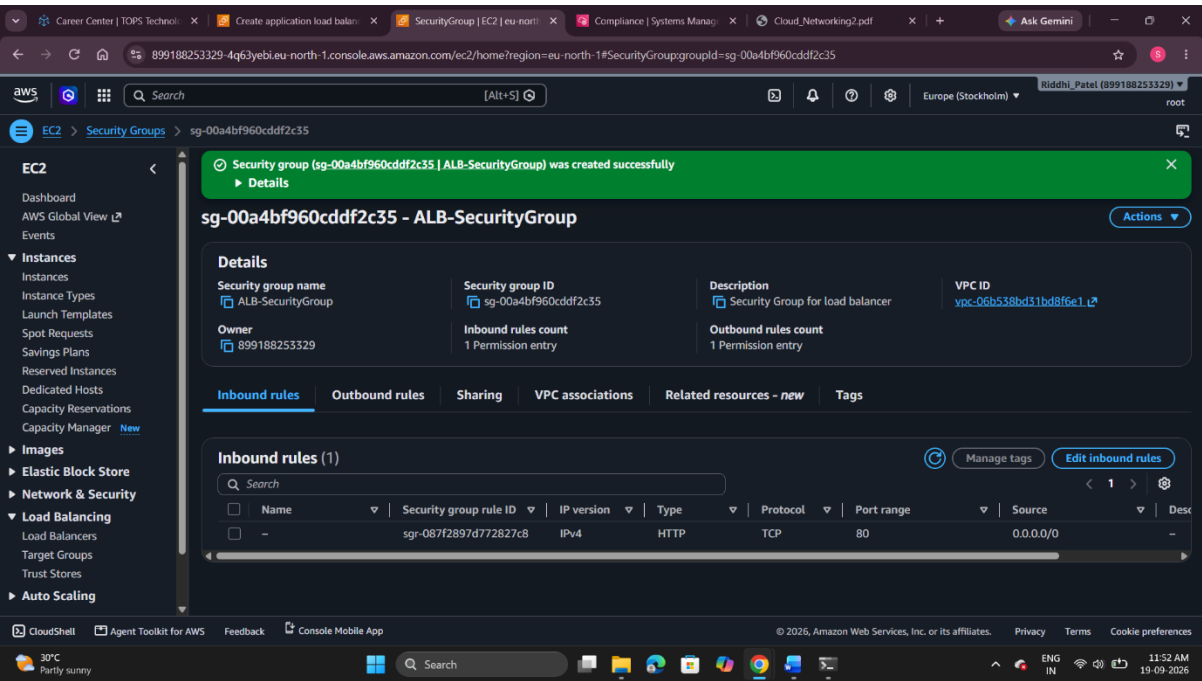


The screenshot displays the AWS Management Console for the 'Webserver-TargetGroup'. The 'Targets' tab is active, showing a table of registered targets. The table has columns for Instance ID, Name, Port, Zone, Health status, Health status details, and Administrative actions. Two targets are listed: 'WebServer-1' and 'WebServer-2', both with a 'Healthy' status. The 'Health status details' column shows a green checkmark and the text 'Healthy'.

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative actions
i-09241b88a4cb2fc57	WebServer-1	80	eu-north-1a (e...)	Healthy	-	-
i-003ba4c3a8675bc2e	WebServer-2	80	eu-north-1a (e...)	Healthy	-	-

Step 2 : The ALB security group allows incoming HTTP traffic on port 80 from the internet.

Screenshot 2: ALB Security Group – HTTP Port 80



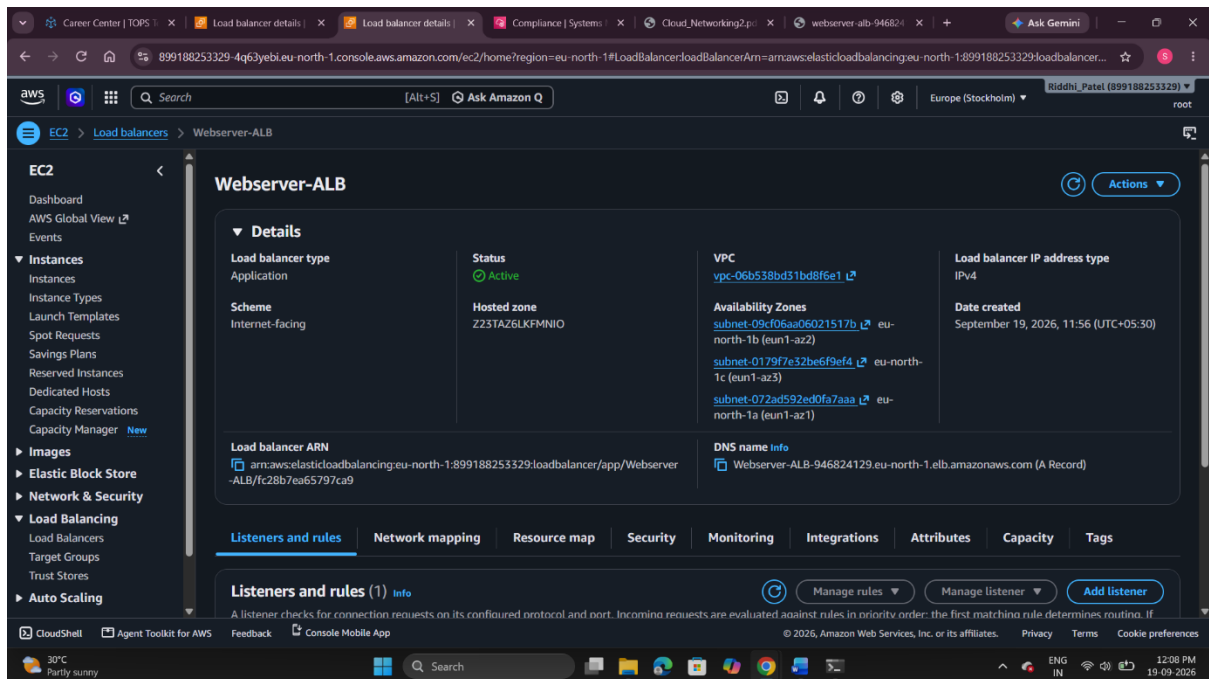
The screenshot displays the AWS Management Console for the 'sg-00a4bf960cddf2c35 - ALB-SecurityGroup'. The 'Inbound rules' tab is active, showing a table of inbound rules. The table has columns for Name, Security group rule ID, IP version, Type, Protocol, Port range, Source, and Destination. One rule is listed: 'sgr-087f2897d77282c8', which allows HTTP traffic on port 80 from 0.0.0.0/0.

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Destination
-	sgr-087f2897d77282c8	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Step 3: Configure Application Load Balancer

I created an **Application Load Balancer** named **Webserver-ALB** with an **Internet-facing** scheme. The ALB uses an HTTP listener on port 80 and forwards requests to the target group.

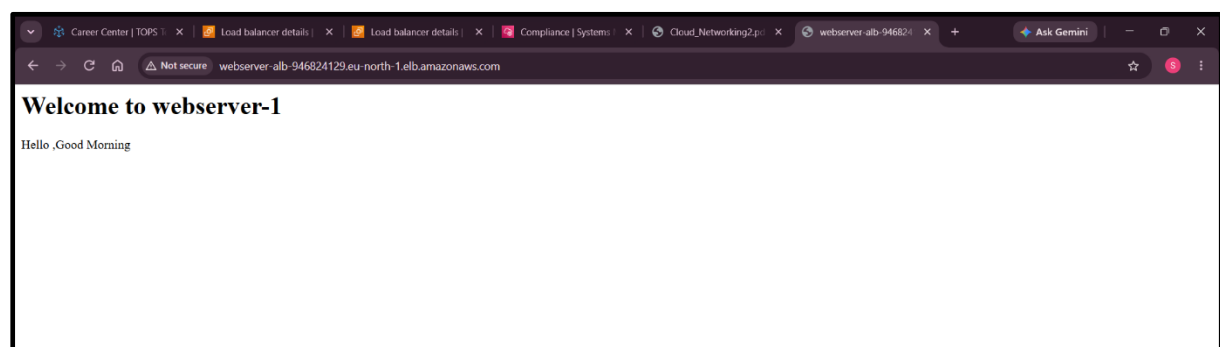
Screenshot 3 — Application Load Balancer Configuration



Step 4: Test WebServer-1 through ALB

I opened the ALB DNS name in the browser using HTTP.

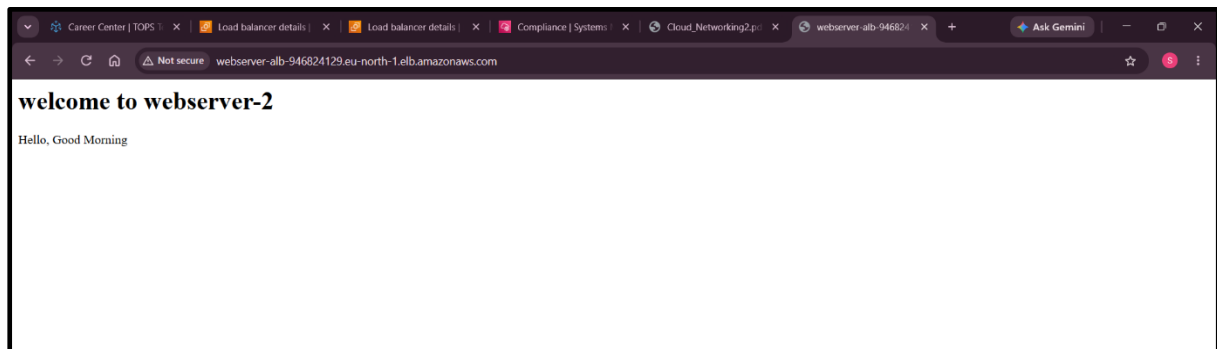
Screenshot 4: ALB Traffic Served by WebServer-1



Step 5: Test WebServer-2 through ALB

The same ALB DNS name was tested again and the request was served by the second EC2 instance.

Screenshot 5: ALB Traffic Served by WebServer-2



Result: The Application Load Balancer was successfully configured with two EC2 instances as healthy targets. The ALB DNS name successfully served responses from both WebServer-1 and WebServer-2.

-
- 3. Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances. Hint: Create a /health endpoint on your web server that returns a 200 OK response.**

Ans:

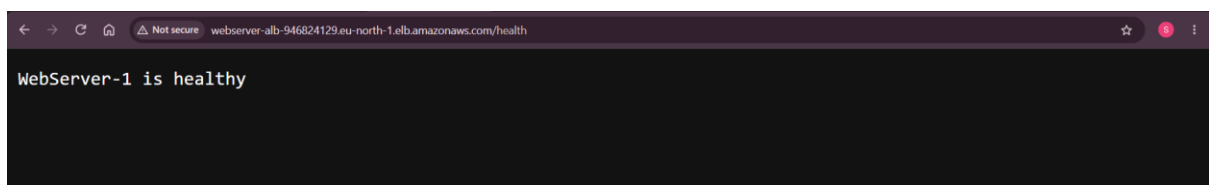
I configured a custom health check path /health for the Application Load Balancer. A /health endpoint was created on both EC2 web servers, and it returned a successful response. The ALB was then configured to use /health for health checks. Finally, one web server was stopped temporarily to verify that the ALB marked it as unhealthy and routed traffic only to the healthy instance.

Step 1: Create /health Endpoint

A /health endpoint was created on both web servers to check their availability.

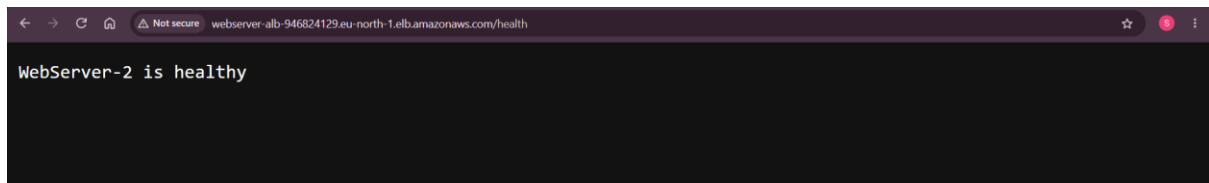
Screenshot 1: WebServer-1 /health Endpoint

The /health endpoint of WebServer-1 returned “WebServer-1 is healthy”, confirming that the server is working properly.



Screenshot 2: WebServer-2 /health Endpoint

The /health endpoint of WebServer-2 returned “WebServer-2 is healthy”, confirming that the server is working properly.

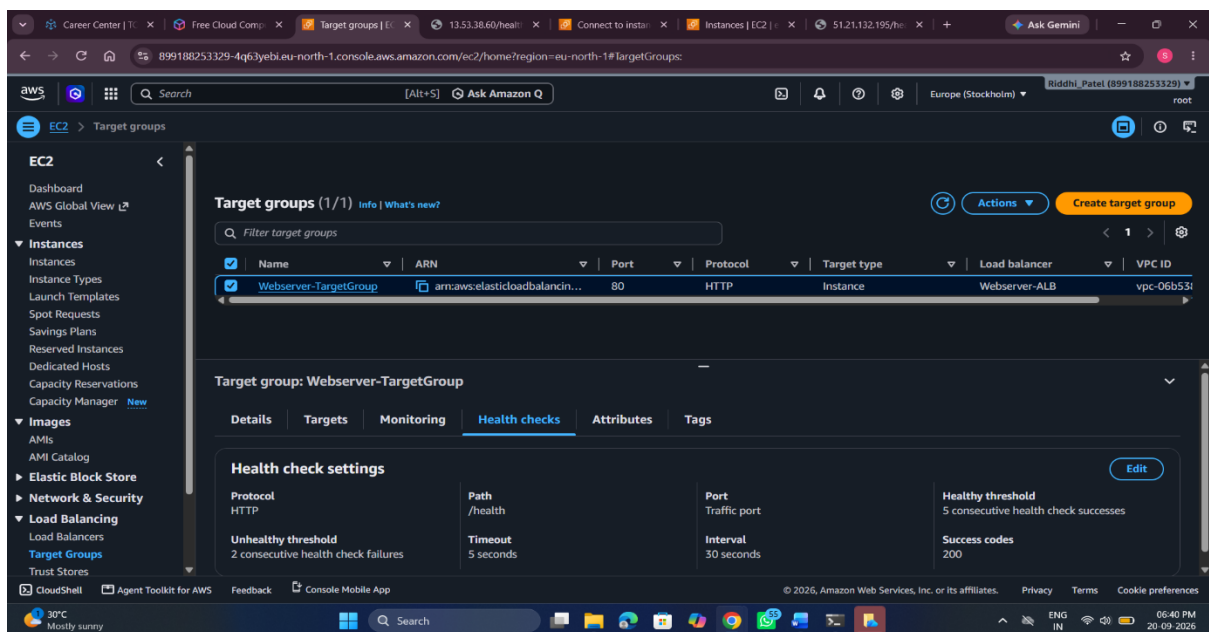


Step 2: Configure ALB Health Check

The health check path of the target group was changed to /health with HTTP protocol and success code 200.

Screenshot 3: ALB Health Check Configuration

The ALB now checks the /health path of each registered EC2 instance. A 200 OK response is considered healthy.

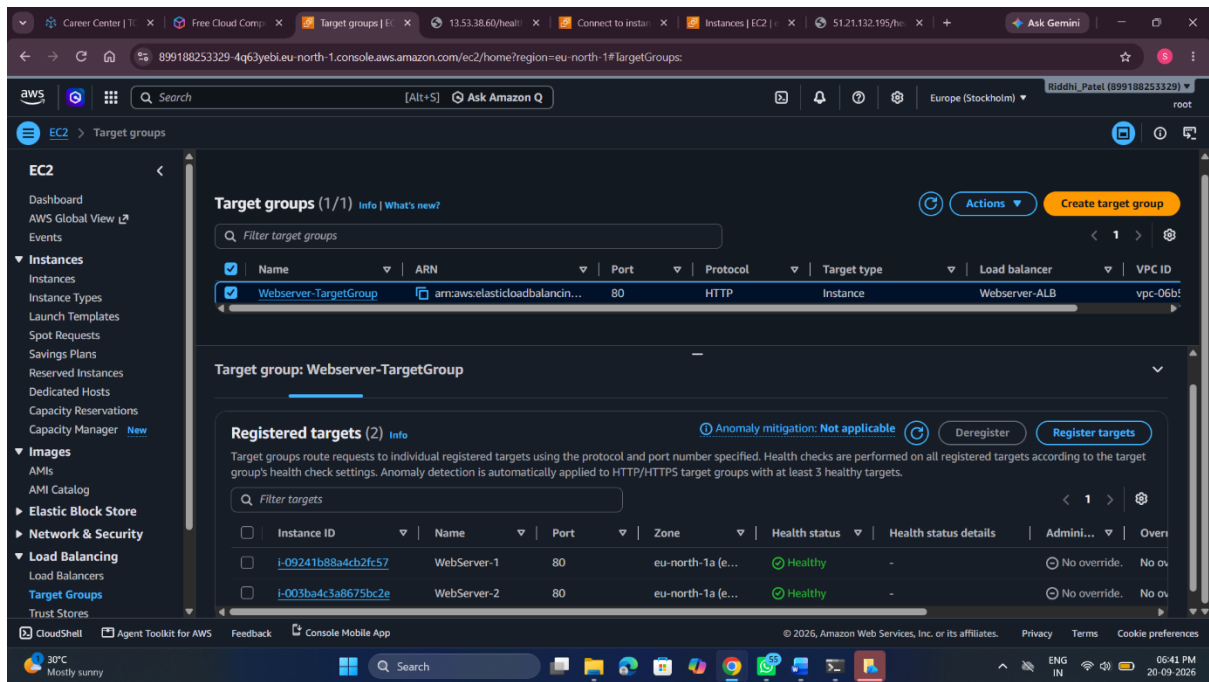


Step 3: Verify Healthy Instances

After configuring the custom health check, both EC2 instances were shown as healthy.

Screenshot 4: Both EC2 Instances Healthy

WebServer-1 and WebServer-2 are both marked Healthy, which means both servers are successfully passing the /health check.

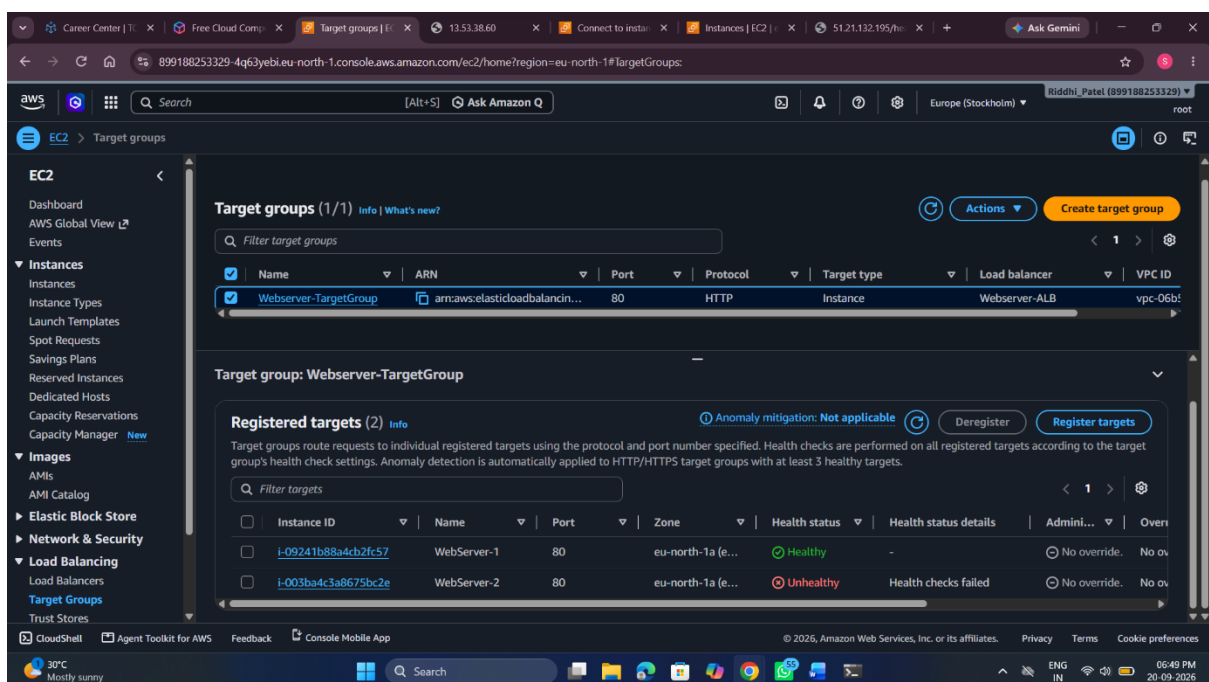


Step 4: Verify Unhealthy Instance Handling

WebServer-2 was temporarily stopped to test the health check. The ALB detected that its /health endpoint was no longer available and marked WebServer-2 as **Unhealthy**.

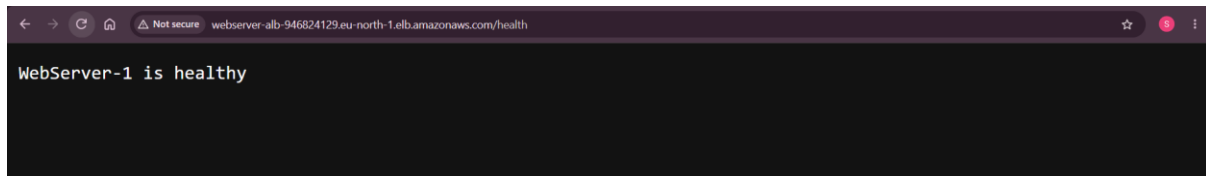
Screenshot 5: Unhealthy Target Detection

WebServer-1 remained **Healthy**, while WebServer-2 became **Unhealthy** because its health checks failed. Therefore, the ALB can route traffic only to the healthy WebServer-1.



Step 5 : Finally, the ALB DNS name was opened in the browser. Since WebServer-1 was healthy and WebServer-2 was unhealthy, the ALB forwarded the request only to WebServer-1. Even after refreshing the page, the response from WebServer-1 was displayed because the unhealthy WebServer-2 was removed from traffic routing.

Screenshot 6 : ALB Routes Traffic Only to Healthy Server



Result:

The custom /health check was successfully configured. The ALB detected the health status of both EC2 instances and stopped routing traffic to the unhealthy instance.

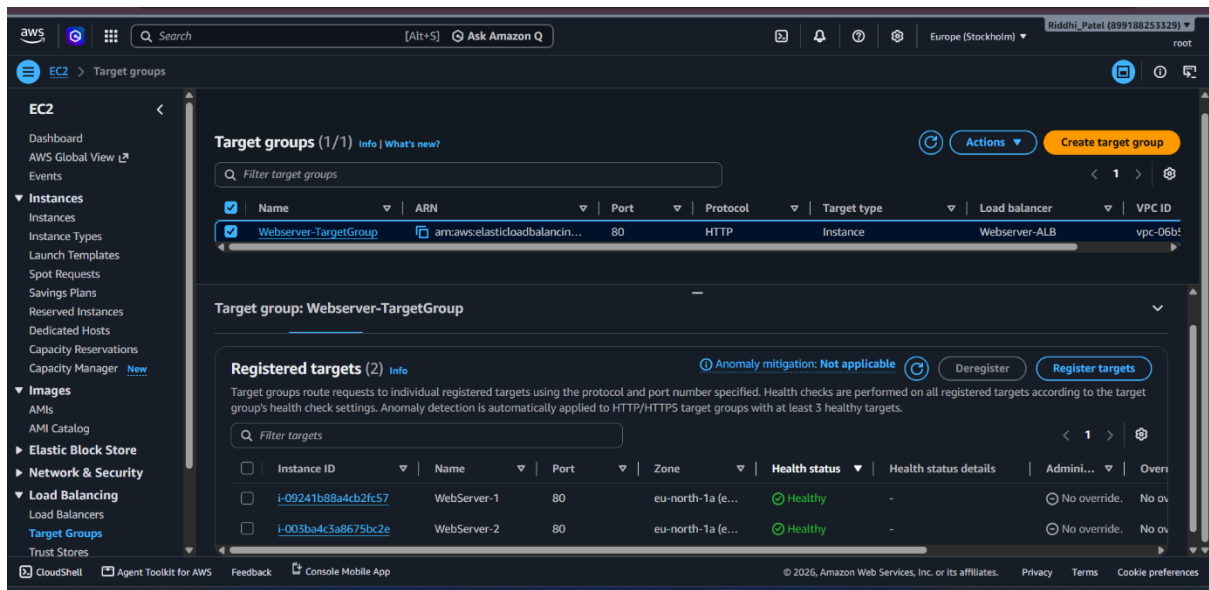
-
- 4. Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.**

Ans: I simulated a server failure by stopping WebServer-2 while keeping WebServer-1 running. The ALB detected that WebServer-2 was no longer available, while WebServer-1 remained healthy. The ALB continued sending requests to the available healthy server, so the website was still accessible. This demonstrates high availability, because the application can continue working even when one server is stopped.

Step 1: Check both servers before stopping one

Before simulating the failure, I checked the Target Group and confirmed that both WebServer-1 and WebServer-2 were healthy.

Screenshot: Both EC2 instances healthy before failure



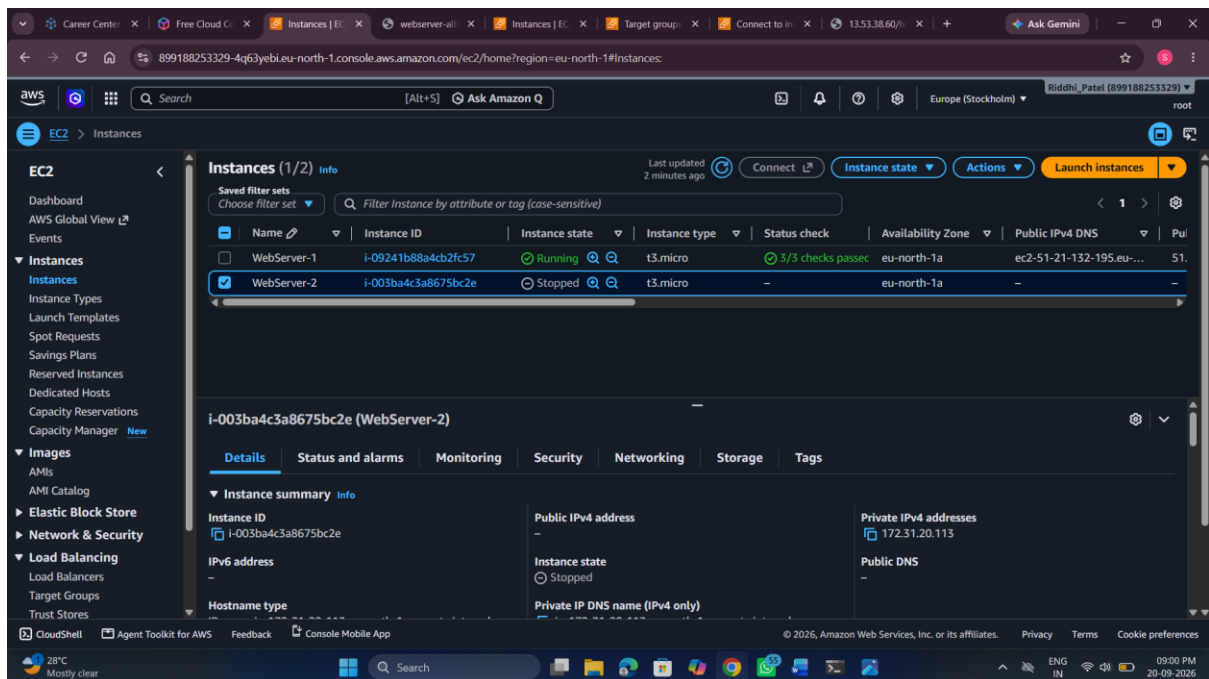
Step 2: Stop WebServer-2

I selected **WebServer-2** from the EC2 Instances page and selected:

Instance state → Stop instance

This simulated a server failure.

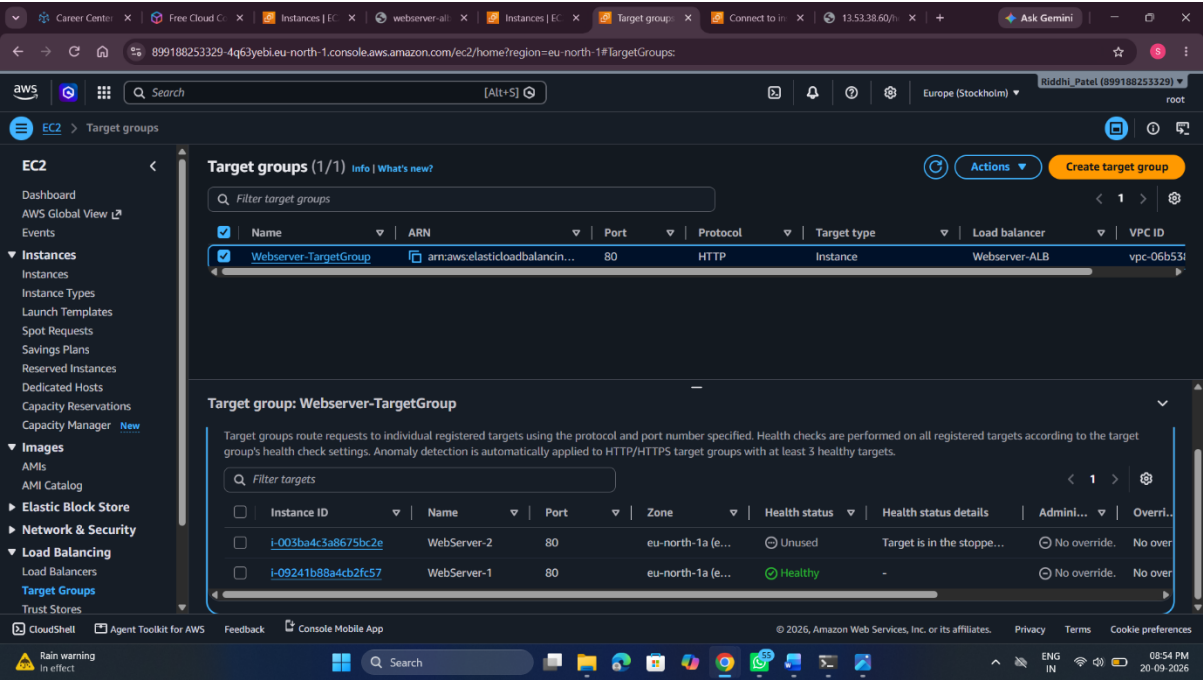
Screenshot: WebServer-2 stopped



Step 3: Check the Target Group

After stopping WebServer-2, I checked the Target Group again. **WebServer-1 remained Healthy**, while **WebServer-2 was shown as Unused/Stopped** because the EC2 instance was no longer running.

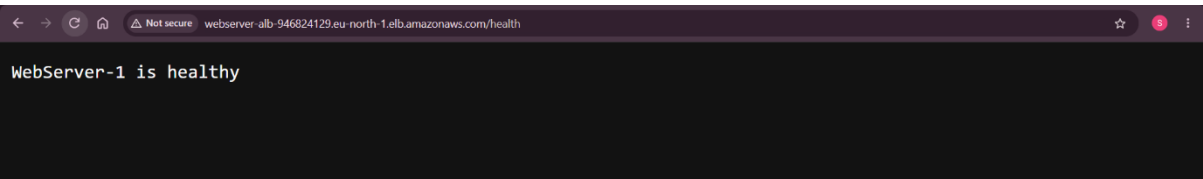
Screenshot: WebServer-1 Healthy and WebServer-2 Unused/Stopped



Step 4: Test the ALB DNS

I opened the ALB DNS name in the browser after stopping WebServer-2. The request was served by WebServer-1, which was still healthy and available.

Screenshot: ALB serving the request from WebServer-1



5. Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

Ans: AWS documentation describes ALB as suited to HTTP/HTTPS and advanced request routing, while NLB is designed for transport-level traffic such as TCP, UDP and TLS. Classic Load Balancer is the previous-generation ELB.

Point	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Supported Protocols	HTTP, HTTPS	TCP, UDP, TLS, QUIC	HTTP, HTTPS, TCP, SSL
OSI Layer	Layer 7 – Application Layer	Layer 4 – Transport Layer	Layer 4 and Layer 7
Main Use Case	Web applications, APIs, microservices and containers	High-performance applications, TCP/UDP traffic and very low latency	Basic load balancing for older EC2-based applications
Routing	Can route based on URL path, host name and HTTP headers	Routes traffic based mainly on IP address and port	Basic routing using connection/request-based methods
Important Features	Advanced routing, HTTP/2, WebSockets, health checks	Very high performance, static IP support, health checks	Basic load balancing, SSL termination and health checks
Best Suited For	Modern web applications	High-speed and network-level applications	Legacy applications

ALB is mainly used for modern web applications and APIs. **NLB** is used for high-performance and TCP/UDP applications. **CLB** is mainly used for older or legacy applications.